

Placement Portal V1 - Project Report

Student details

Name - Abhay Sahu

Roll no. - 23f1001119

Email - abhaysahu7355@gmail.com

About me - I am a Computer Science student pursuing a BS in Data Science and Applications from IIT Madras. I have a strong interest in software development, machine learning, and data-driven applications. I enjoy solving real-world problems by designing scalable systems and building web applications. Through my academic journey, I have gained experience in Python, Flask, SQLAlchemy, SQLite, HTML/CSS, Bootstrap, VueJS. I am always eager to learn new technologies and apply them to practical projects that improve efficiency and user experience.

Project Description

The Placement Portal V1 is a multi-user web application designed to streamline the entire campus recruitment process for students, companies, and the institute placement cell. The system features three distinct user roles: an Admin (Placement Cell) with full system control, Companies that register and post placement drives, and Students who browse and apply for those drives.

The application allows students to register, log in, view approved placement drives, apply for them, track their application status, and maintain a placement history. Companies can register (pending admin approval), create placement drives, view applicants, and update application statuses. The admin oversees and manages all users, companies, drives, and applications from a centralized dashboard.

How I approached the problem statement

I began by carefully reading the project requirements and breaking them into three core workflows — one for each role. I then sketched the data model on paper before writing any code.

I prioritized building a clean database schema first, carefully designing relationships between the five core tables (Admin, Company, Student, PlacementDrive, Application) to ensure data integrity. A UniqueConstraint on (student_id, drive_id) was added at the database level to prevent duplicate applications.

I then built the backend using Flask with all routes organized in a single routes.py file divided into five sections: helpers, auth, admin, company, and student. A login_required(role) decorator was written to protect every dashboard route, ensuring strict role isolation.

For the frontend, I designed a unified auth system — a single login.html and register.html handles all three roles using a ?role= URL parameter, keeping the code clean and non-repetitive. Dashboard pages extend a shared layout_dashboard.html with a fixed navbar and sidebar. All CSS is consolidated in a single styles.css file.

Finally, I tested all three user flows end-to-end — admin approval workflow, company drive creation and applicant management, and student registration and application — before finalising the submission.

My approach to implementing this project was structured as follows:

- 1. Requirement Analysis:** Analyzed the project requirements to identify the three roles, five core entities, and all required workflows before writing any code.
- 2. Database Design:** Designed a 5-table schema with proper foreign keys, cascade deletes, and a UniqueConstraint on (student_id, drive_id) to prevent duplicate applications.
- 3. Backend Development:** Built all routes in a single routes.py file with a custom login_required(role) decorator protecting every dashboard route.
- 4. Unified Auth System:** Built one login.html and one register.html for all three roles instead of six separate pages, passing role as a URL parameter to keep code DRY.
- 5. User Interface Development:** Developed 27 HTML templates extending a shared base.html and layout_dashboard.html, with all CSS consolidated into a single styles.css file.
- 6. Security Implementation:** Implemented password hashing (bcrypt via Werkzeug), role-based access control, ownership verification for company drives, and PDF file upload validation.
- 7. Testing:** Conducted thorough testing of all three user flows to ensure all core functionalities work correctly and reliably.

DB Schema Design

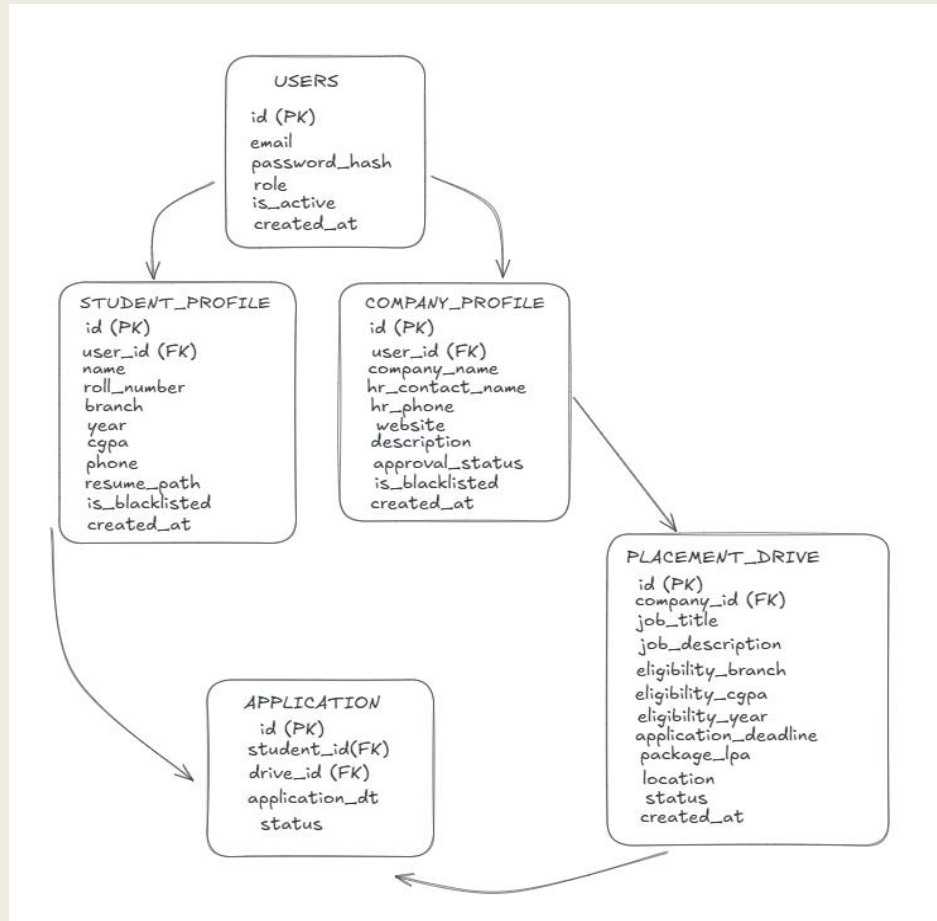
The Placement Portal database consists of five interconnected tables to manage all placement-related information. The database is created programmatically using SQLAlchemy models — no manual DB creation tools are used.

The Admin table stores the pre-seeded superuser account (seeded automatically on first run). The Company table holds company profiles with an approval_status field (pending / approved / rejected / blacklisted) that controls login access. The Student table stores student details with an is_active flag that allows admin to deactivate accounts.

The PlacementDrive table links to Company via a foreign key and has a status field (pending / approved / closed / rejected) — only approved drives are visible to students. The Application table links both Student and PlacementDrive via foreign keys and has a UniqueConstraint on (student_id, drive_id) to prevent any student from applying to the same drive more than once. Application status tracks each stage: applied, shortlisted, selected, or rejected.

All relationships use cascade delete — deleting a company removes all its drives, and deleting a drive removes all applications for that drive.

ER Diagram



Frameworks and Libraries Used

Core Technologies

- **Flask** : Lightweight backend framework for building web applications with Python
- **SQLite**: Database management system for storing application data
- **SQLAlchemy**: ORM (Object-Relational Mapping) tool for all database interactions
- **Jinja2**: Template engine for rendering dynamic HTML views

Frontend

- **Bootstrap** : Frontend framework for responsive design and UI components
- **HTML5 / CSS3**: Page structure, layout and custom styling
- **Bootstrap Icons** : Icon library used throughout the application

Extensions

- **Werkzeug** : Password hashing using bcrypt and secure file utilities

API Resource Endpoints

Auth APIs

- GET / — Home page with role selection cards
- GET/POST /login?role=admin|company|student — Unified login for all three roles
- GET/POST /register?role=company|student — Register a company or student
- GET /logout — Clear session and redirect to home

Admin APIs

- GET /admin/dashboard — Dashboard with stats (students, companies, drives, applications)
- GET /admin/companies — View all companies with filter tabs
- GET /admin/companies/<id>/approve — Approve a company registration
- GET /admin/companies/<id>/reject — Reject a company registration
- GET /admin/companies/<id>/blacklist — Blacklist a company
- GET /admin/students — View all students
- GET /admin/students/<id>/deactivate — Deactivate a student account
- GET /admin/drives — View all placement drives with filter tabs
- GET /admin/drives/<id>/approve — Approve a placement drive
- GET /admin/drives/<id>/reject — Reject a placement drive
- GET /admin/search — Search students and companies by name or ID

Company APIs

- GET /company/dashboard — Dashboard with created drives and applicant counts
- GET/POST /company/drives/create — Create a new placement drive
- GET/POST /company/drives/<id>/edit — Edit an existing drive
- GET /company/drives/<id>/close — Close a drive to stop new applications
- GET /company/drives/<id>/delete — Delete a drive
- GET/POST /company/drives/<id>/applications — View applicants and update their status
- GET/POST /company/profile/edit — Edit company profile

Student APIs

- GET /student/dashboard — Dashboard with stats, applied drives and status
- GET /student/drives — Browse all approved open placement drives
- GET /student/drives/<id> — View drive detail page
- POST /student/drives/<id>/apply — Apply for a drive
- GET /student/applications — View own application history
- GET/POST /student/profile/edit — Edit profile and upload PDF resume

Key Features Implemented

Core Features

- Three-role authentication system (Admin / Company / Student) with a unified login and register page
- Admin dashboard with live stats — total students, companies, drives, and applications
- Company approval workflow — companies register as pending, admin approves before login is allowed
- Placement drive approval workflow — drives go through pending → approved before students can see them
- Company can create, edit, close, and delete placement drives
- Company can view all applicants per drive and update status (shortlisted / selected / rejected)
- Students can browse approved drives, apply, and track application status
- Duplicate application prevention at both route level and database level (UniqueConstraint)
- Admin can blacklist or deactivate company and student accounts
- Admin can search students and companies by name or ID
- Student profile editing with PDF resume upload (5MB max, validated file type)
- Profile dropdown in navbar — avatar + name + role with links to dashboard, edit profile, and logout
- Role-based access control — every dashboard route protected by `login_required(role)` decorator
- Ownership verification — companies can only edit/delete their own drives

Challenges and Solutions

Challenge 1 : Single Login System for Three Roles

Solution: Instead of building six separate login and register templates (one per role per action), a single `/login` and `/register` route was built. The role is passed as a URL parameter (`?role=admin/company/student`). Jinja2 conditionals in the template handle showing the correct icon and label. The route uses `if/elif` blocks to query the correct database table.

Challenge 2 : Preventing Duplicate Applications

Solution: A SQLAlchemy UniqueConstraint was added on (`student_id`, `drive_id`) in the Application model. The apply route also checks for an existing application before inserting, so the student sees a clear flash message rather than a database error.

Challenge 3 : Preventing Cross-Company Data Access

Solution: Every company route that accesses a drive verifies `drive.company_id == company.id` before allowing any operation. If the IDs do not match (e.g. a company guessing another company's drive ID in the URL), the request is rejected with a 403 error.

Challenge 4 : `url_for()` Parameter Syntax Errors

Solution: A `BuildError` occurred because query parameters were appended outside `url_for()` like `url_for('auth_login')+"?role=admin"`. This was corrected to `url_for('auth_login', role='admin')` which passes the role as a proper keyword argument inside Flask's URL builder.

Challenge 5 : CSS Not Loading Correctly

Solution: The home page appeared unstyled because Google Fonts CDN was timing out. Fixed by moving the font import inside the CSS file using `@import` and adding system font fallbacks (Inter, -apple-system, BlinkMacSystemFont, Segoe UI) so the page renders correctly even without internet access.

Conclusion

The Placement Portal V1 application successfully fulfills all the core requirements specified in the project statement. The system implements a clean three-role architecture with proper session-based authentication, role-based access control, and data integrity constraints at both the application and database level.

Future enhancements could include email notifications when application status changes, interview scheduling functionality, resume parsing to auto-extract student skills, placement analytics charts using Chart.js, and a mobile-responsive collapsible sidebar.

Demonstration Link -

<https://drive.google.com/file/d/1-itu-KxyMO5EwudB6J-Lp5HH9EQxJVUh/view?usp=sharing>